UNIVERSITY OF SZEGED
Department of Software Engineering
UNIVERSITAS SCIENTIARUM SZEGEDIENSIS

Webfejlesztési keretrendszerek

6. gyakorlat

Dr. Jánki Zoltán Richárd

Mi lehet a problémák gyökere?

- A felhasználó hozzáad egy receptet, de a katalógusban nem jelenik meg, csak frissítést követően.
- A felhasználó félig kitölti a új recept hozzáadása űrlapot, majd elnavigál, visszanavigál, de az űrlap üres.
- A “Mentés” gombra kattintva az adat kétszer kerül elküldésre, de néha egyszer sem.

A legtöbb bug…

- Nem UI hibák fordulnak elő a legtöbb esetben
- Nem API hibák a leggyakoribbak

- Állapotkezelési hibák szinte kivétel nélkül

- Az állapotkezelés nem szintkatikai kérdés, hanem **tervezési döntés** → *itt tud az AI is a legtöbbször hibázni.*

Mi az állapot?

- Állapot (state) = minden olyan adat, ami az alkalmazás adott pillanatbeli viselkedését meghatározza.
- 3 típusa
  - UI állapot
  - Szerver állapot
  - Űrlap állapot

- Nem minden adat állapot!
  - Statikus szöveg vagy kép ≠ állapot
- State az, ami változhat a felhasználó interakciója mentén

Állapot típusok scope szerint

- Lokális
  - egyetlen komponenshez kötődik
  - kívülről nem kell látni
  - pl.: dropdown állapota (nyitva/zárva), input aktuális értéke
- Globális (megosztott)
  - több komponens is függ tőle
  - több ponton kell hozzáférni
  - pl.: bejelentkezett felhasználó adatai, bevásárlólista tartalma, téma beállítás
- Tervezési döntés:
  - minden state legyen a legalacsonyabb szinten

Single Source of Truth (SSOT)

- Minden adatnak pontosan egy igazság forrása van
  - az adatot nem tároljuk két helyen
  - az adat egy helyen megváltozik, innen olvassa mindenki

Típus	Igazság forrás	Receptgyűjtemény alkalmazásban minta
UI állapot	Komponens	Hamburgermenü nyitva/zárva
Szerver állapot	Adatbázis	Heti menü slot-ok tartalma
Űrlap állapot	FormGroup	Recept létrehozás űrlap

Állapotkezelési technikák

Állapotkezelési technikák

- Input-Output direktívákkal
  - adatátadás hierarchiában egymáshoz közel álló komponensek között
- Szolgáltatásokkal (service-ekkel)
  - adatátadás egymástól távol levő komponensek között
  - Reaktív programozással
  - Signal-okkal

- Anti-pattern: Globális állapot túlzott használata

Állapot “felemelése” (state hoisting)

- Ha az adat egyetlen komponensben él, tartsuk ott (lokálisan)
- Ha az adat több (max 1-2) egymásnak alárendelt komponensben él, alkalmazzunk szülő-gyerek kommunikációt
- Ha két azonos szinten levő (testvér-) komponensnek ugyanaz az adat kell, az állapot kerüljön egy közös szülőbe
- Ha nem közvetlen rokon komponens használja (vagy route váltás után is meg kell tartani), használjunk szolgáltatást

UNIVERSITAS SCIENTIARUM SZEGEDIENSIS
UNIVERSITY OF SZEGED
Department of Software Engineering

Reaktív programozás (RxJS)

- Adatfolyam kezelés aszinkron módon
- Push-alapú működés
- Az adatfolyamra feliratkozunk (subscribe) és leiratkozunk (unsubscribe)

- Potenciális memory-leak(!) jelenség
  - ha nincs leiratkozás, a komponens nem szüntethető meg
  - memóriában marad, és nem tud később sem megszűnni
  - a komponens megszűnésekor (destroy esemény) le kell iratkozni az adatfolyamokról

Signal-ok

- A reaktív programozás (RxJS megoldásainak) egyszerűsítése
  - több fejlesztői feladat volt (pl.: feliratkozás, leiratkozás)
  - szinkron megoldás
  - pull-alapú megoldás
- signal()
  - reaktív érték
- computed()
  - származtatott érték (újraszámolódik)
- effect()
  - mellékhatás végrehajtása

Űrlapok kezelése

- Sablon-alapú megközelítés
  - egyszerűbb implementálni
  - hamarabb korlátokba ütközhetünk
  - Angular-ban: ngModel
- Reaktív űrlapok
  - logikája a komponens osztályban van
  - könnyebben ellenőrizhető, a logika egy helyen van
  - típusbiztos
  - tesztelhető
  - komplex validációs technikák
  - Angular-ban: FormGroup, FormControl, FormArray

Gyakorlat

- Alkossuk meg az űrlapokat
  - Login
  - Receptfelvétel
  - Profilkezelés
- Kezeljük helyesen az állapotokat
  - Menü és dialógusablak állapotok
  - Adatok átküldése komponensek között
  - Lekérdezések és szinkronizációk